Technical and Reproducibility Appendix for Concrete Strength Modeling

Abby Zhang, Emily Shi

Algorithmic Design & Implementation Details

We wrote our own batch gradient descent to fit linear models for concrete compressive strength. The same function works for univariate models ($y = mx + b$) and for multivariate models with $p = 8$ features ($p + 1$ parameters).

Objective function. For n training samples, the model predicts $\hat{y}_i = x_i^\top w + b$, where w is the weight vector and b is the bias. We minimize the mean squared error:

$$J(w,b) = \frac{1}{n}\sum_{i=1}^{n}\left(x_i^\top w + b - y_i\right)^2 \tag{1}$$

Gradient derivation. Let $e_i = x_i^\top w + b - y_i$ be the error of sample i. By the chain rule, the derivative of e_i^2 is $2e_i x_i$ for w and $2e_i$ for b. Taking the average over all n samples gives the gradient we use in each update, where X is the $n \times p$ feature matrix and e is the vector of errors:

$$\nabla_w J = \frac{1}{n}\sum_{i=1}^{n} 2e_i x_i = \frac{2}{n}X^\top e \tag{2}$$

$$\frac{\partial J}{\partial b} = \frac{1}{n}\sum_{i=1}^{n} 2e_i = \frac{2}{n}\sum_{i=1}^{n} e_i \tag{3}$$

Update rule. In each iteration, both parameters move one step against the gradient, with learning rate α:

$$w \leftarrow w - \alpha\nabla_w J, \qquad b \leftarrow b - \alpha\frac{\partial J}{\partial b} \tag{4}$$

We evaluate every model with MSE and with variance explained, $R^2 = 1 - \mathrm{MSE}/\mathrm{Var}(y)$.

Pseudocode.

```
function GRADIENT_DESCENT(X, y, alpha, iterations = 20000):
    if X has one column: reshape X to n x 1
    w <- zeros(p);  b <- 0
    losses <- [ mean((X w + b - y)^2) ]
    repeat iterations times:
        e      <- X w + b - y
        grad_w <- (2 / n) * X^T e
        grad_b <- 2 * mean(e)
        w <- w - alpha * grad_w
        b <- b - alpha * grad_b
        loss <- mean((X w + b - y)^2)
        if loss is not finite: stop with error (alpha too large)
        append loss to losses
    return w, b, losses
```

Hyperparameters and stopping criteria. All weights and the bias start at 0. The **maximum number of iterations is 20,000** for every model, and every run uses all 20,000 iterations. The only other stopping rule is a safety check such that if the loss becomes infinite or NaN, the function stops with an error, because this means α is too large. We used **fixed learning rates** that gave stable training. For the raw and standardized multivariate models, we chose the rates using a stability limit calculated from the training data. **All the rates we used are below their stability limits** (Table 1).

Model	Learning rate α	Approximate stability upper bound
Untransformed multivariate	3×10^{-7}	5.99×10^{-7}
Standardized multivariate	0.2	0.427
Log-transformed multivariate	0.003	0.0057
Univariate models (8)	5×10^{-7} to 5×10^{-3}	–

Table 1: Learning rates used for gradient descent. Training is stable only when α is strictly below the upper bound; the bound itself is not included.

Software environment. We used Python 3 with NumPy for matrix calculations and the gradient descent loop, pandas and xlrd for reading `Concrete_Data.xls`, scikit-learn (`LinearRegression`) for the optimal solutions, statsmodels (`OLS`) for p-values, and Matplotlib for the loss plots. We reproduced all results with Python 3.10.12, NumPy 2.2.6, pandas 2.3.3, scikit-learn 1.7.2, statsmodels 0.15.0, Matplotlib 3.10.9 and xlrd 2.0.2.

Dataset partition. The dataset has 1,030 samples, 8 input features (seven ingredients in kg/m^3 and age in days) and one target, compressive strength in MPa. We split it into four steps:

1. Read `Concrete_Data.xls` with pandas and keep the original row order.
2. Use rows 501–630 (Python indices 500–629) as the **testing set: 130 samples**.

3. Join rows 1–500 and 631–1030 as the **training set: 900 samples**.

4. Use the first 8 columns as X and the last column as y.

For standardization, the mean and standard deviation come from the training set only. The testing set is never used to update the model.

Feature Scaling & Gradient Descent Dynamics

Scaling & Convergence Speed

Approach. We used our gradient descent to train two multivariate models for 20,000 iterations. One uses the raw features. The other uses standardized features, where each feature is changed with its training mean μ_j and standard deviation σ_j:

$$z_j = \frac{x_j - \mu_j}{\sigma_j} \tag{5}$$

Results. Both models got a positive training R^2 within the first five iterations. However, they reached the best training MSE (104.14, from `LinearRegression`) at very different speeds (Table 2 and Figure 1). The standardized model was within 1% of the best MSE after only **79 iterations**, while the raw model needed **5,335 iterations**, so **scaling made training about 70 times faster**. The raw model was still a little above the best MSE after 20,000 iterations.

Model	α	Iterations to reach 1% of best MSE	Training MSE after 20,000	Training R^2	Testing R^2
Untransformed	3×10^{-7}	5,335	104.52	0.623	0.377
Standardized	0.2	79	104.14	0.624	0.368

Table 2: Gradient descent with raw and standardized features.

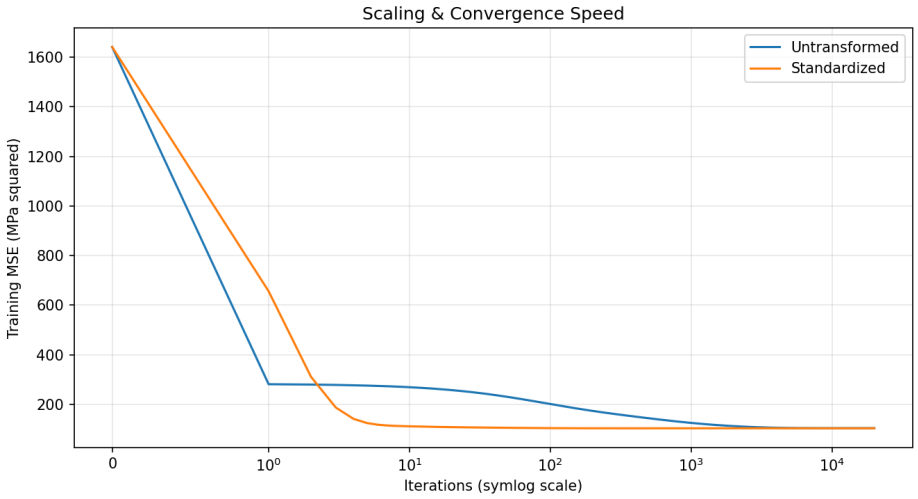


Figure 1: Training MSE over 20,000 iterations for raw ($\alpha = 3 \times 10^{-7}$) and standardized ($\alpha = 0.2$) features. The x-axis uses symmetric log scale.

Why. The raw features have very different ranges. For example, coarse aggregate is around 974 kg/m^3, but superplasticizer is around 7 kg/m^3. Because of this, the loss surface is very steep in some directions and very flat in others. The learning rate must be small enough for the steep directions (below about 5.99×10^{-7}), or the loss explodes. With such a small step, the flat directions move very slowly. After standardization, every feature has mean 0 and standard deviation 1, so the surface is much more even and the stable rate can be much larger, so we used $\alpha = 0.2$. Both rates we used are about half of their upper bounds, and both models showed a stable, decreasing loss. So **scaling allows a much larger stable learning rate and makes convergence much faster**.

Scaling & The Optimal Solution

Approach. We used `LinearRegression` to find the exact optimal solution for the raw and the standardized multivariate models. Then we changed the standardized coefficients back to raw units:

$$w_j^{\text{raw}} = \frac{w_j^{\text{std}}}{\sigma_j}, \qquad b^{\text{raw}} = b^{\text{std}} - \sum_j \frac{w_j^{\text{std}} \mu_j}{\sigma_j} \tag{6}$$

Results. The two optimal models have the **same** training MSE (104.14), testing MSE (141.31), training R^2 (0.624) and testing R^2 (0.368). Their test predictions differ by at most 7×10^{-14} MPa, which is only rounding error. Table 3 shows the coefficients.

Feature	Raw model	Standardized model	Standardized, changed back to raw units
Cement	0.1324	14.032	0.1324
Blast furnace slag	0.1247	10.481	0.1247
Fly ash	0.1061	6.817	0.1061
Water	−0.1341	−2.986	−0.1341
Superplasticizer	0.1172	0.701	0.1172
Coarse aggregate	0.0285	2.213	0.0285
Fine aggregate	0.0338	2.770	0.0338
Age	0.1188	7.281	0.1188
Intercept	−51.429	36.927	−51.429

Table 3: Optimal coefficients from `LinearRegression` for raw and standardized features.

A linear transformation does not change the optimal solution in a meaningful way. The coefficients look different only because the units are different such that one is per kg/m^3 (or per day), the other is per standard deviation. After we change them back, they are exactly the same. Standardization only shifts and rescales each feature, so both models can make the same predictions and have the same minimum.

Linear vs. Non-Linear Feature Transformations

Approach. We applied the transformation below to every feature and trained a new multivariate model with our gradient descent ($\alpha = 0.003$, 20,000 iterations). We use $\log(x+1)$ instead of $\log(x)$ because many mixes have 0 for some ingredients (44% have no blast furnace slag, 52% no fly ash and 31% no superplasticizer), and $\log(0)$ is not defined.

$$x'_j = \log(x_j + 1) \tag{7}$$

Convergence speed and stability. After the log transformation, all features are between 0 and 7.04. Their ranges are much closer, so we could use a learning rate **10,000 times larger** than for the raw features without the loss exploding. The training R^2 became positive after just one iteration. But the end of training was slow. It took 12,257 iterations to get within 5% of the best MSE (55.83), and after 20,000 iterations the training MSE was still 56.94 (Figure 2). One reason is that the log features are not centered at 0, so changing the weights and changing the bias have similar effects on the loss, and the two are hard to separate. In summary, **scaling and the log transformation both allow larger stable learning rates**. All three models showed a stable, decreasing loss with the settings we chose, while **standardization made convergence much faster** than the log transformation.

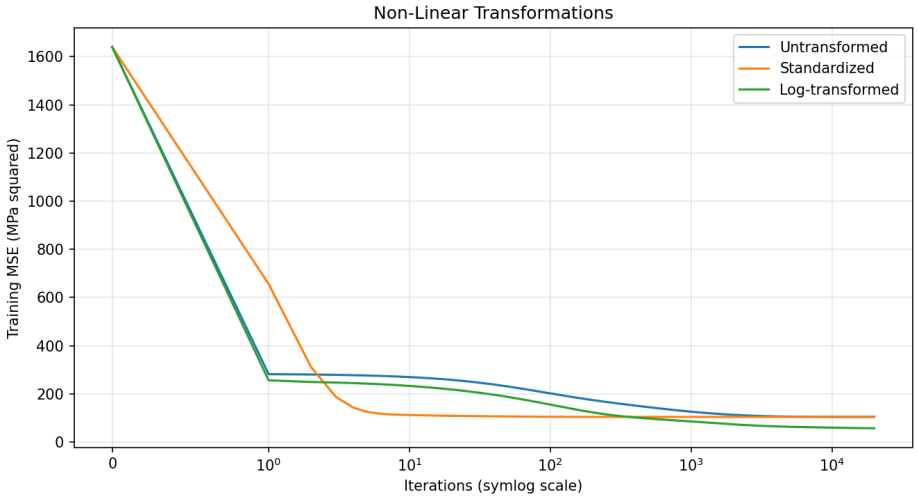


Figure 2: Training MSE over 20,000 iterations, with the log-transformed model added ($\alpha = 0.003$). The x-axis uses symmetric log scale.

Optimal models. We then used `LinearRegression` to find the optimal solution for each version of the features. Table 4 compares the three optimal models.

Optimal model	Training MSE	Testing MSE	Training R^2	Testing R^2
Untransformed	104.14	141.31	0.624	0.368
Standardized	104.14	141.31	0.624	0.368
Log-transformed	55.83	54.38	0.798	0.757

Table 4: Summary of the three optimal models from `LinearRegression`.

Linear vs. non-linear. Standardization is linear, so the model can make the same predictions as before, and the optimal model does not change. The log transformation is non-linear. The model is still a straight line in $\log(x)$, but it is a curve in the original x: when a feature gets larger, each extra unit has a smaller effect. This fits concrete well. For example, concrete gets stronger quickly in the first weeks and then slows down, and the log changes age from 1–365 days to 0.69–5.9. Because of this, **testing R^2 goes up from 0.368 to 0.757**. The log model also works better on new data. Its testing MSE (54.38) is close to its training MSE (55.83), but for the raw model the testing MSE (141.31) is much higher than the training MSE (104.14).

Univariate vs. Multivariate Model Dynamics

Approach. For each of the 8 raw features, we used our gradient descent to train $y = m_i x_i + b$ for 20,000 iterations, with a different learning rate for each feature. Then we used `LinearRegression` to find the optimal solution for each univariate model and for the full 8-feature model (Table 5).

Feature	Mean ± SD	α	GD training R^2	Optimal training R^2	Univariate coefficient	Multivariate coefficient
Cement	277.9 ± 105.9	5×10^{-6}	0.173	0.264	0.0808	0.1324
Blast furnace slag	76.1 ± 84.0	3.5×10^{-5}	−0.482	0.025	0.0316	0.1247
Fly ash	57.4 ± 64.2	6×10^{-5}	−0.180	0.042	−0.0534	0.1061
Water	180.7 ± 22.3	1.5×10^{-5}	−0.216	0.077	−0.2070	−0.1341
Superplasticizer	6.9 ± 6.0	5×10^{-3}	0.102	0.102	0.8881	0.1172
Coarse aggregate	973.8 ± 77.5	5×10^{-7}	−0.076	0.016	−0.0274	0.0285
Fine aggregate	772.3 ± 81.9	8×10^{-7}	−0.124	0.023	−0.0306	0.0338
Age	46.0 ± 61.3	8×10^{-5}	0.082	0.123	0.0952	0.1188

Table 5: Univariate models on the 900 training samples. Age is in days, and the other features are in kg/m^3. GD means our gradient descent after 20,000 iterations.

Easy and hard models. Three of the eight models reached a positive training R^2 with gradient descent. **Superplasticizer was the easiest.** Its values are small, so we could use the largest learning rate (0.005), and it reached its optimal R^2 of 0.102. Cement and age also became positive, but they did not reach their optimal R^2. The other five models were still negative after 20,000 iterations. **Water, coarse aggregate and fine aggregate were the hardest**, because their mean is 8 to 13 times larger than their standard deviation. Large values force a very small learning rate, but the bias needs to move far. For example, the optimal bias for coarse aggregate is 63.6, but gradient descent only moved it to 0.008.

The coefficients are not the same. Superplasticizer drops from 0.888 in its univariate model to 0.117 in the multivariate model. Blast furnace slag goes up from 0.032 to 0.125. Fly ash, coarse aggregate and fine aggregate even **change sign**, which means they are negative alone, but positive in the multivariate model.

Feature correlations and omitted variable bias. In the multivariate model, each coefficient shows the effect of one feature when the other seven features stay the same. A univariate model only sees one feature, so it cannot separate that feature from the others. If a missing feature affects strength and is correlated with the feature in the model, its effect gets mixed into the coefficient. This is the **omitted variable bias**, and it only goes away when the features are not correlated. In our training set, the features are clearly correlated:

- Fly ash and cement have a correlation of -0.48, so mixes with more fly ash usually have less cement. Less cement makes concrete weaker, so the univariate model blames fly ash and gives it a negative coefficient.
- Superplasticizer and water have a correlation of -0.66, so mixes with more superplasticizer usually have less water. The univariate superplasticizer coefficient is too large because it also includes the benefit of using less water.

Statistical Inference & Regression Analysis

Approach. We used statsmodels `OLS` on the 900 training samples to get the p-value of each feature, for the raw, standardized and log-transformed features. We added the intercept with `sm.add_constant`. Each p-value tests if that coefficient could be 0 (Table 6).

Feature	Untransformed	Standardized	Log-transformed
Cement	2.5×10^{-40}	2.5×10^{-40}	1.0×10^{-85}
Blast furnace slag	6.4×10^{-26}	6.4×10^{-26}	7.0×10^{-36}
Fly ash	3.3×10^{-13}	3.3×10^{-13}	**0.292**
Water	0.0018	0.0018	1.7×10^{-20}
Superplasticizer	**0.243**	**0.243**	8.4×10^{-6}
Coarse aggregate	0.0056	0.0056	**0.405**
Fine aggregate	0.0030	0.0030	**0.089**
Age	8.6×10^{-74}	8.6×10^{-74}	1.8×10^{-192}

Table 6: Feature p-values from statsmodels `OLS` (900 training samples). Bold values are not significant at the 0.05 level.

Standardization does not change significance. The p-values of all eight features are exactly the same for raw and standardized features. Standardization is linear, so each coefficient and its standard error are multiplied by the same number, and their ratio (the t-statistic, which gives the p-value) stays the same. **The log transformation changes significance a lot**, because it creates a different model (training R^2 goes from 0.624 to 0.798). Superplasticizer becomes significant (p goes from 0.243 to 8.4×10^{-6}), while fly ash, coarse aggregate and fine aggregate are no longer significant. Age, which was already the most significant feature, becomes even more significant.

Important predictors vs. easy or hard univariate models. There is only a weak trend. Cement and age are the two most significant predictors, and both of their univariate models reached a positive R^2. But superplasticizer was the easiest univariate model, and it is the only feature that is not significant in the raw model. Blast furnace slag and fly ash are very significant, but their univariate models were hard to train. This is because **how easy a model is to train depends on the size and range of the feature values**, which decide the learning rate, while **significance depends on how much a feature helps to predict strength** when the other features are also in the model.

Code & Reproducibility Access

All code and results are in our GitHub repository:
`https://github.com/abbyzhang0918/CSE5104-Project1`
To reproduce all results:

1. Download the repository:
```
git clone https://github.com/abbyzhang0918/CSE5104-Project1.git
cd CSE5104-Project1
```

2. Install the packages:
```
python3 -m pip install numpy pandas xlrd scikit-learn statsmodels matplotlib
```

3. Run our scripts:

```
python3 code/gradient_descent_optimization.py
python3 code/scaling_convergence_speed.py
python3 code/scaling_optimal_solution.py
python3 code/non_linear_transformations.py
python3 code/univariate_vs_multivariate.py
python3 code/statistical_significance.py
```

What each script does. (Figures and tables are saved in the `results/` folder.)

- `gradient_descent_optimization.py`
  Data split, gradient descent, MSE and R^2. Results are printed.
- `scaling_convergence_speed.py`
  Scaling & Convergence Speed. Saves `scaling_convergence.png`.
- `scaling_optimal_solution.py`
  Scaling & The Optimal Solution. Results are printed.
- `non_linear_transformations.py`
  Non-Linear Transformations. Saves `nonlinear_convergence.png` and `optimal_models.csv`.
- `univariate_vs_multivariate.py`
  Univariate vs. Multivariate Coefficients. Saves `univariate_gradient_descent.csv` and `univariate_vs_multivariate_coefficients.csv`.
- `statistical_significance.py`
  Statistical Significance. Saves `p_values.csv`.

There is no randomness in our code, so running the scripts again gives the same numbers.

Use of AI tools. We used Claude to help check parts of the code and improve report writing.